

Large Language Model (LLM)-powered Autonomous Agents and Multiagent System

COS30018-Intelligent Systems (Swinburne Vietnam)

Hoang Huu Hoan, 105732188

Pham Ho Quang Dung, 105706604

Nguyen Nam Khanh, 105706565

Tran Manh Duc, 104774105

Semester May 2026

Deadline: 09/08/2026

Instructor: Dr. Nguyen Manh Toan

Repository: https://github.com/TranDuc511/COS30018_Group-4.1_Multi-Agent-System-for-Restaurant-Review-Analysis

Contents

1. Introduction	4
1.1 Background and Motivation	4
1.2 Project Goals (G1 and G2)	4
1.3 Mission and Scope	4
2. Overall System Architecture	5
2.1 Architecture Overview	5
2.2 Agent Roles and Responsibilities	6
2.3 Orchestration and Recovery (LangGraph)	7
2.4 Structured Data Contracts (Pydantic)	7
2.5 API and Frontend	9
2.6 External Libraries and Software Packages	10
3. Installation and User Guide	11
3.1 Prerequisites	11
3.2 Backend Setup	11
3.3 Frontend Setup	12
3.4 Running the System (CLI / Web / Server)	12
3.5 Building the SQLite Index	12
4. Implementation Details	12
4.1 Member 1. Hoang Huu Hoan:	12
4.2 Member 2. Pham Ho Quang Dung:	13
4.3 Member 3. Nguyen Nam Khanh:	14
4.4 Member 4. Tran Manh Duc:	14
5. Experimentation and Evaluation	14
5.1 Evaluation Methodology	15
5.2 Tier 1 — Deterministic Checks	15
5.3 Tier 2 — Analysis Gold Set	15
5.4 Tier 3 — LLM-as-Judge (Rubrics)	16
5.5 Latency and Cost	16
6. Demonstration Scenarios: End-to-End Report for a Restaurant	17
6.1 User flow on how to use the web	17
6.2 Report generated from the web	18
7. Critical Analysis of the Implementation	21
7.1 Strengths	21
7.2 Limitations and Known Issues	21
7.3 Security and Deployment Considerations	22
7.4 Design Trade-offs	22

8. Discussion of Experimental Results	22
8.1 Interpretation of Results	22
8.2 Threats to Validity.....	23
9. Beyond The Class Project Scope.....	23
10. Summary and Conclusion.....	23
10.1 Summary of Achievements	23
10.2 Future Work	24

1. Introduction

1.1 Background and Motivation

Restaurant feedback is scattered across large volumes of unstructured review text. A restaurant owner reading Yelp reviews one by one cannot reliably extract what recurs, what is anecdotal, and what is actionable. The Option D brief frames this as the Business Improvement Agent mission: turn raw customer reviews into evidence-linked insight — recurring positive and negative patterns, cautious root-cause hypotheses, and prioritised improvement actions — rather than a bare sentiment count.

Large language models make this tractable without task-specific training: LLMs can perform aspect-based sentiment analysis by prompting alone [14][15][16], and decomposing a complex task across several role-specialised LLM agents is now an established design paradigm [1][2][3][4][5]. This project applies both ideas to the Yelp Open Dataset, with an explicit emphasis on making every claim the system produces either machine-verified, recomputed, or explicitly bounded.

1.2 Project Goals (G1 and G2)

- **G1 — a single LLM-powered autonomous agent.** Achieved. Each agent in the system (most visibly the Analysis Agent) is an autonomous LLM unit: it plans its output against a fixed JSON contract, validates its own result with Pydantic, and self-corrects from validation feedback for up to two retries before reporting a structured error — a post-hoc self-correction loop in the sense of [6][7][8][9].
- **G2 — a collaborating multi-agent system.** Achieved. Four specialised agents (Analysis, Reasoning, Strategy, Report) collaborate through typed contracts under a central supervising orchestrator implemented in LangGraph. The orchestrator runs after every stage, measures each output with deterministic checks, and decides whether the pipeline proceeds, proceeds with a warning, retries the stage with feedback, or halts.

Both goals were carried through to a live, evaluated system: the full pipeline was run end-to-end against real Yelp data with a four-tier evaluation harness (Section 5).

1.3 Mission and Scope

Input: reviews for one selected restaurant from the Yelp Open Dataset (local copy, ~5.34 GB review file, indexed into SQLite). **Output:** a structured business-improvement report — sentiment and aspect summary, recurring patterns with evidence review IDs, root-cause hypotheses with confidence levels, prioritised recommendations, and an explicit limitations statement — delivered through a FastAPI backend and a React dashboard with live pipeline progress.

Scope boundary, stated plainly: the system samples **at most 100 reviews per restaurant** (a validated, seeded random sample; configurable 1–100 per request) and does **not** claim to analyse the whole Yelp dataset or the whole review history of a business. Non-goals: production deployment (the API is a local-demo API — Section 7.3), multilingual analysis, and real-time review ingestion.

2. Overall System Architecture

2.1 Architecture Overview

The system is a staged pipeline wrapped by a supervising hub. Data flows from the Yelp dataset through business matching, review loading and preprocessing, then through four LLM agents; after **every** agent stage the orchestrator node measures the output and routes the graph. Delivery is via FastAPI (synchronous POST or Server-Sent Events streaming) into a React dashboard.

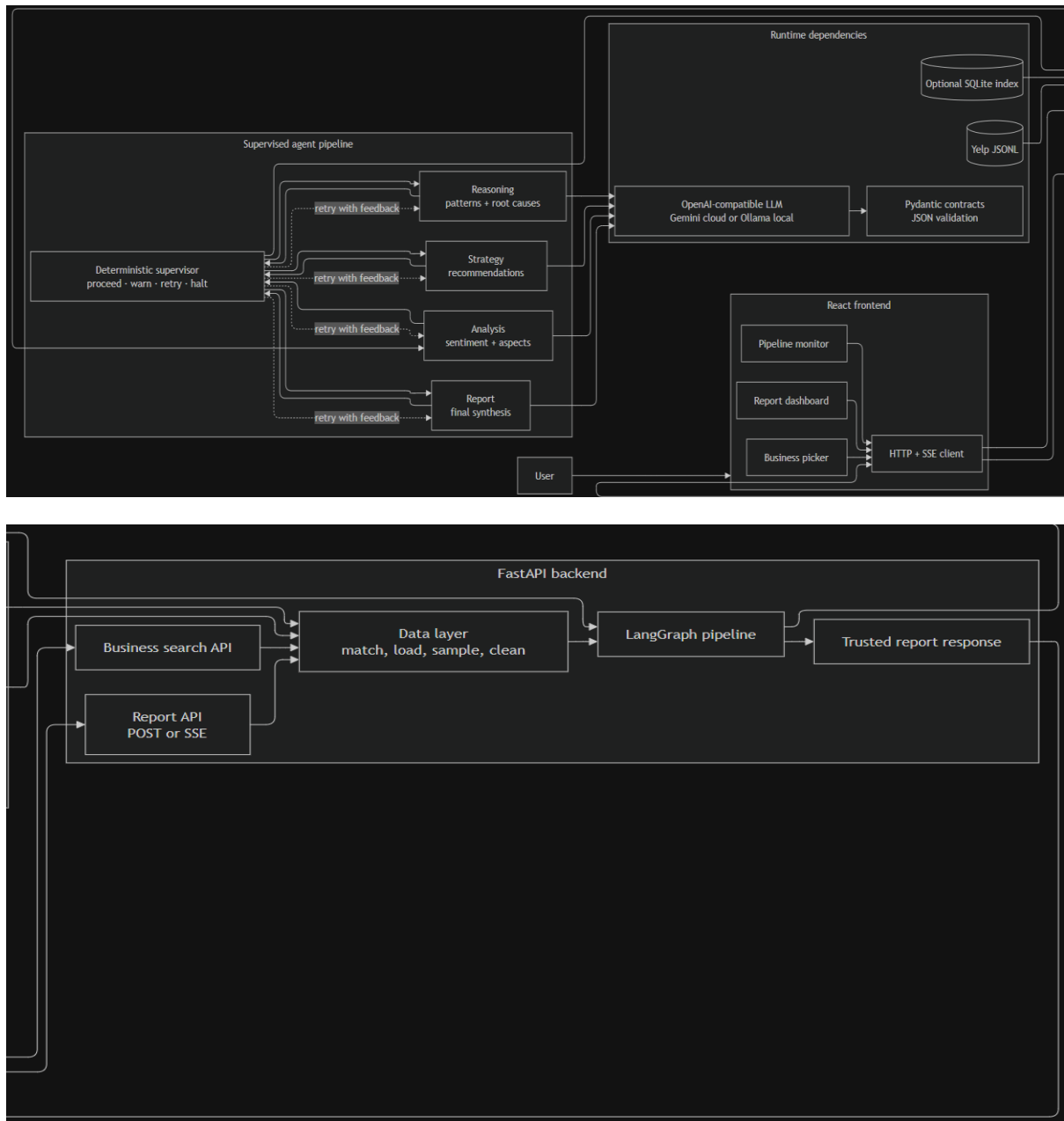


Fig 1.1 – 1.2: System architecture diagram

Fig 1.1 – 1.2 describes the major runtime components of the restaurant-review analysis system and how they communicate. A user begins in the React frontend, which provides a business picker, report dashboard, and pipeline monitor. These interfaces share an HTTP/SSE client that communicates with the FastAPI backend. Business-search requests are routed to the search API, while report-generation requests use either a standard POST endpoint for a complete response or

an SSE endpoint for live pipeline updates. The backend's data layer resolves the selected restaurant, loads reviews from raw Yelp JSONL files or an optional SQLite index, performs seeded random sampling, and cleans the selected records. The sample is capped at 100 reviews by default, meaning the system analyses a reproducible subset rather than the restaurant's entire Yelp history.

The cleaned reviews enter the LangGraph multi-agent pipeline, which contains four specialised agents. The Analysis Agent assigns sentiment and extracts review aspects; the Reasoning Agent identifies recurring patterns and evidence-backed root causes; the Strategy Agent converts those findings into prioritised recommendations; and the Report Agent synthesises the results into the final restaurant report. A deterministic supervisor runs after every agent. It validates the stage result and chooses whether to proceed, proceed with a warning, retry the same agent with feedback, or halt the pipeline. Therefore, the conceptual sequence is Analysis → Reasoning → Strategy → Report, but the actual runtime sequence always passes through supervision between stages. Retry feedback returns to the responsible agent without restarting the entire pipeline.

The agents access an OpenAI-compatible LLM interface, allowing the same pipeline to use Gemini in the cloud without changing agent logic. Every LLM response must pass through Pydantic contracts, which validate that the output is structured JSON and contains required fields such as status and error_detail. Once processing finishes successfully, FastAPI constructs a trusted response containing the report, summaries, warnings, and pipeline metadata. Important values such as the restaurant identity and actual sample size come from application state rather than being trusted from generated model text. The response then returns to the frontend, where the dashboard displays the report and the pipeline monitor shows progress, verdicts, retries, warnings, or errors.

In short, this diagram answers: which components exist, where responsibilities are located, and how the frontend, backend, data sources, agent pipeline, supervisor, LLM, and contracts work together at runtime.

2.2 Agent Roles and Responsibilities

- **Analysis Agent** (`analysis_agent.py`) — classifies each review's overall sentiment (positive / negative / neutral / mixed) and per-aspect polarity over seven aspect categories (`food_quality`, `staff_attitude`, `pricing`, `wait_time`, `ambience`, `cleanliness`, `other`). Reviews are batched 10 per LLM call; a cross-record validator requires the returned review-ID list to equal the input list in order, and a violation re-enters the same self-correction loop as a schema error. It is the only stage with objectively checkable answers, so it anchors Tier 2 evaluation.
- **Reasoning Agent** (`reasoning_agent.py`) — aggregates the successful analysis rows into recurring negative patterns (description, aspect, frequency, evidence review IDs) and root-cause hypotheses (pattern, cause, confidence low/medium/high). Its claimed pattern frequencies are treated as advisory: the orchestrator recomputes them in Python and overwrites them (Section 2.3).
- **Strategy Agent** (`strategy_agent.py`) — converts patterns and root causes into prioritised recommendations (priority ≥ 1 , issue, action, category, expected impact). It has both an LLM mode (default) and a deterministic fallback mode that ranks by confidence then frequency.
- **Report Agent** (`report_agent.py`) — assembles the final `ReportOutput`: title, business name, sample size, executive summary, key findings, root causes, recommendations, and a mandatory limitations statement. Business name and sample size are overwritten from trusted pipeline state so the model cannot mislabel the report.

- **Orchestrator (hub)** (`core/orchestrator.py` + `core/graph.py` + `core/supervision.py`) — not an LLM agent on the common path: a deterministic supervisor that measures every stage output and routes the graph (Section 2.3).

2.3 Orchestration and Recovery (LangGraph)

The orchestration follows the supervisor idea from multi-agent LLM frameworks [1][2][5], with one deliberate difference: routine routing decisions are made by **hard rules over measured facts, not by an LLM**. The implemented graph (the "simple hub", Option B of the team's two design options) places a single orchestrator node after every agent stage:

```
preprocess -> analysis -> ORCH -> reasoning -> ORCH -> strategy -> ORCH -> report -> ORCH
-> END
```

- **Measure, then decide.** `supervision.measure(stage, state)` computes deterministic facts from pipeline state — evidence-ID resolution, recomputed pattern frequencies, a stars-vs-sentiment contradiction rate, sample-size floors, recommendation traceability, and report-subset consistency. `supervision.decide(stage, facts, retry_counts)` then applies hard rules with fixed thresholds (`MIN_VIABLE_N = 5`, `LOW_CONFIDENCE_N = 30`, `CONTRADICTION_MAX = 0.20`, `FREQUENCY_TOLERANCE = 0.05`, `TRACE_SIMILARITY = 0.35`).
- **Verdict vocabulary:** `proceed` | `proceed_with_warning` | `retry` | `halt`. The earlier design's skip verdict was found to be self-contradictory during design review (it could silently drop the stage after the failed one) and was **dropped**: a non-critical stage that exhausts retries is recorded in `skipped_agents` with a `gave_up_after_retries` flag and the pipeline proceeds with a warning instead.
- **Critical vs non-critical.** Analysis and Reasoning are critical — unrecoverable failure halts the pipeline cleanly rather than emitting a hollow report. Strategy and Report failures degrade to on-record warnings where safe (Report exhaustion halts, since without it there is nothing to deliver).
- **Two bounded correction loops share one mechanism.** Inside each agent, Pydantic validation failures trigger up to two correction re-prompts carrying the validation error verbatim. At graph level, a retry verdict writes the measured reason into `state["retry_feedback"]` (e.g. "evidence id X does not exist in the analysis output"), which the agent node appends to the prompt on re-run. This matters because agents run at temperature 0: re-running an identical prompt reproduces the identical wrong output, so a retry without feedback is a mechanically wasted retry. Retries are capped at 2 per stage; the recursion limit is raised to 50 because a worst-case run (2 retries × 4 stages) is exactly LangGraph's default limit of 25 node visits.
- **Frequency is authoritative in Python.** On a reasoning `proceed`, the orchestrator overwrites each pattern's LLM-claimed frequency with the value recomputed from the analysis rows, flagging any divergence beyond ± 0.05 . LLM numeric claims are thereby demoted to advisory — a concrete defence against fabricated statistics.

A legacy LLM-assisted recovery method (`decide_recovery`) remains in the orchestrator class but is not on the live routing path; the team explicitly chose a fully deterministic supervisor for auditability and reproducibility. Section 7.5 discusses why this combination is one of the project's distinctive contributions.

2.4 Structured Data Contracts (Pydantic)

Every inter-agent payload is a Pydantic v2 model in `backend/app/schemas/contracts.py`. The chain of contracts is:

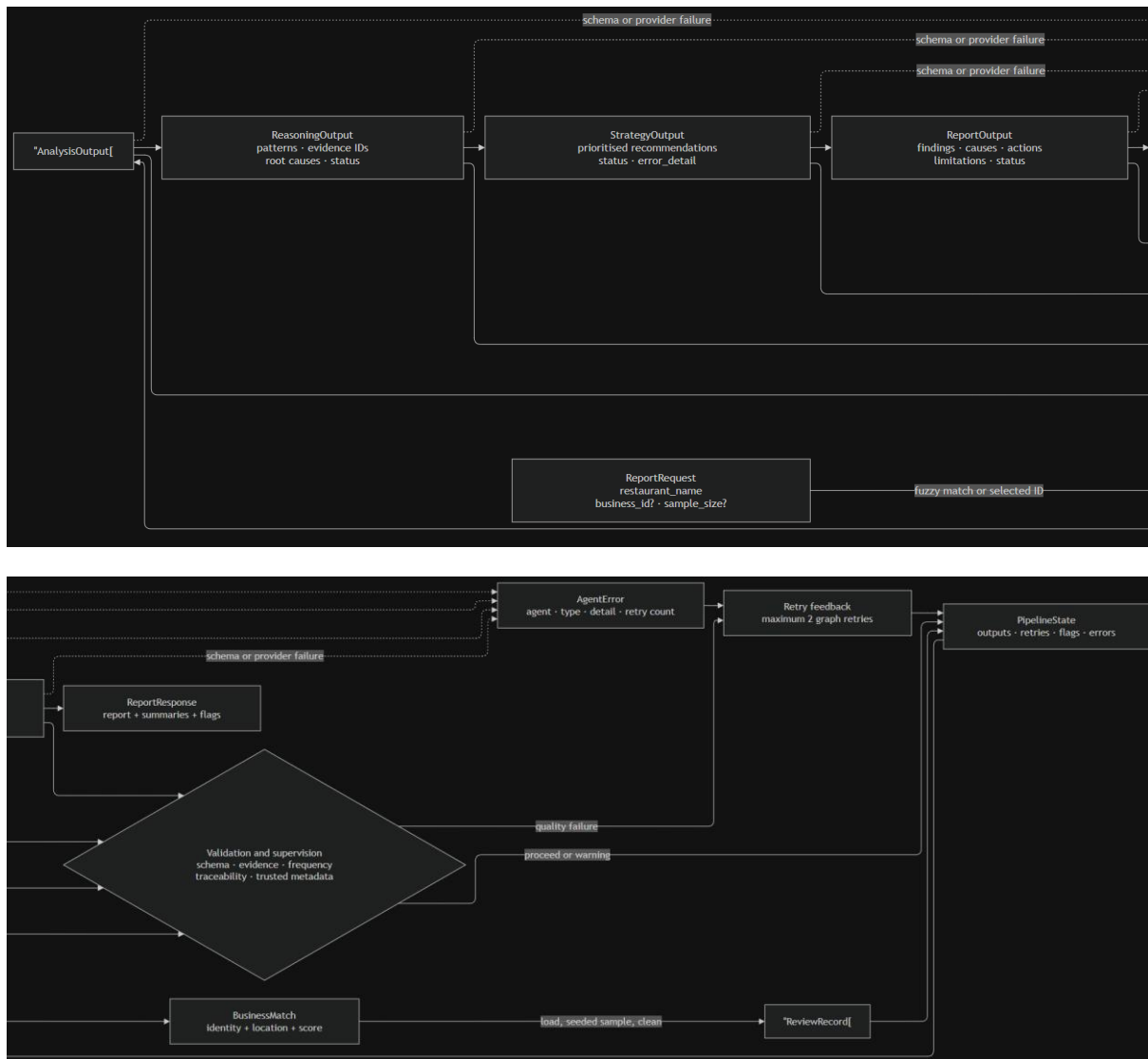


Fig 2.1-2.2: Structural Data contract diagram

Fig 2.1-2.2 focuses on how information changes shape as it travels through the system. It starts with a `ReportRequest`, which contains the restaurant name and may also contain a selected `business_id` and requested `sample_size`. The backend resolves this request into a `BusinessMatch`, representing the restaurant's trusted identity, location, and matching score. When a business ID has already been selected by the frontend, the backend can use that exact identity; otherwise, it performs fuzzy matching against the available business records.

The matched restaurant is used to load a collection of `ReviewRecord` objects. Each review contains a review ID, business ID, star rating, text, and date. Before agent processing, the data layer validates and cleans these records, removes unusable or duplicate entries, applies deterministic ordering, and selects a seeded random sample. This ensures that the same request and seed can reproduce the same review set. The sample cannot exceed the configured maximum of 100, and the final report must describe the result as an analysis of the selected sample rather than all Yelp reviews.

The selected reviews are inserted into `PipelineState`, the shared state object used by `LangGraph`. This state carries the original request information, trusted business metadata, sampled reviews, outputs from completed agents, retry counters, warning flags, errors, and routing information. Each agent reads only the information it needs and adds a structured result back into this state.

The stage contracts are:

- `AnalysisOutput[]` contains review-level sentiment, extracted aspects, review identity, processing status, and error details. The outputs must preserve the input review IDs and their order so later evidence can be traced back to the original sample.
- `ReasoningOutput` contains patterns, supporting review IDs, root causes, frequencies, status, and error information. Every evidence ID must refer to a review that was actually analysed. Pattern frequencies are recomputed by deterministic Python logic instead of relying entirely on an LLM-generated number.
- `StrategyOutput` contains prioritised recommendations connected to the problems identified during reasoning. Recommendations must trace back to an existing pattern or root cause rather than introducing unrelated business advice.
- `ReportOutput` contains the final findings, causes, recommended actions, limitations, status, and error details. The report may summarise and reorganise existing results, but it cannot invent new root causes or recommendations that were not produced by earlier stages.

After each output, the result enters **validation and supervision**. Validation is stage-specific: analysis is checked for coverage and consistency; reasoning is checked for valid evidence and accurate frequencies; strategy is checked for traceability; and the report is checked for unsupported claims and trusted metadata. A valid result returns to `PipelineState` with a `proceed` or `warning` decision, allowing the next stage to run.

If the output has acceptable structure but fails a quality or grounding rule, the supervisor creates retry feedback and routes execution back to the same stage. Graph-level quality retries are limited to two attempts. Separately, an individual agent may perform bounded self-correction when the LLM returns invalid JSON or violates its Pydantic schema. If schema correction or provider recovery fails, the problem is represented as an `AgentError`, which records the agent, error type, details, and retry count. The supervisor then decides whether the pipeline can retry, continue with a warning where permitted, or must halt. Critical failures in analysis, reasoning, or final report generation normally prevent a trusted final result.

At the end of the successful flow, `ReportOutput` is converted into `ReportResponse`. This response combines the final report with compact analysis and reasoning summaries, pipeline flags, warnings, and other frontend-facing metadata. Before returning it, the backend overwrites sensitive identity fields—such as restaurant name and actual sample size—with trusted values from the request and pipeline state. This prevents generated content from changing factual execution metadata.

The contract diagram therefore expresses several guarantees:

1. Sampling is bounded, seeded, and reproducible.
2. Every review-level result remains connected to an original review ID.
3. Evidence in reasoning can be traced back to analysed reviews.
4. Recommendations can be traced back to identified causes or patterns.
5. The final report cannot introduce unsupported conclusions.
6. Every LLM stage returns validated structured data or an explicit error.
7. Retries are bounded, preventing an infinite agent loop.
8. Trusted application metadata takes precedence over model-generated text.

2.5 API and Frontend

Method	Path	Parameters	Returns
GET	/health	—	process-level health {"status": "ok"}
GET	/api/businesses/search	name, top_n (default 3)	fuzzy business matches; 404 if none
POST	/api/reports	restaurant_name, business_id?, sample_size? (1–100)	full ReportResponse; HTTP 500 on halt
GET	/api/reports/stream	restaurant_name, business_id?, sample_size?	SSE progress events + final report

The SSE contract reflects the hub: stage_start and stage_end carry a 1-based attempt number (a retried stage re-emits stage_start with attempt 2), a verdict event reports the orchestrator's decision with its flags and reason after every stage, error follows a halt, and done delivers the final report plus accumulated flags so the dashboard can badge low-confidence results. The response also carries analysis_summary computed in plain Python from the analysis rows — sentiment and aspect counts are never LLM-generated.

The frontend is a Vite + React app with two views: a **Dashboard** (business search and confirmation, KPIs, sentiment and aspect summaries, findings, patterns, root causes, recommendations, limitations) and a **Pipeline Monitor** (live SSE stage status and timings). Known gap: the Pipeline Monitor currently keys its timeline by stage alone and does not yet render the new verdict / attempt events — an additive UI change that remains open (Section 7.2).

2.6 External Libraries and Software Packages

Table 1a — Backend (from backend/requirements.txt; versions unpinned — see Section 7.2):

Library	Version	Purpose
fastapi	unpinned	HTTP API framework (endpoints, SSE)
uvicorn[standard]	unpinned	ASGI server
pandas	unpinned	review DataFrame handling / preprocessing
pydantic	unpinned (v2)	structured inter-agent contracts + validation
python-dotenv	unpinned	load .env provider configuration
rapidfuzz	unpinned	fuzzy business-name matching
openai	unpinned	OpenAI-compatible LLM client (Gemini / Ollama / Groq)
langchain / langchain-openai	unpinned	LLM framework base; ChatOpenAI for legacy recovery path
langgraph	unpinned	multi-agent pipeline graph
scikit-learn	unpinned	Tier 2 accuracy + macro-F1
pytest	unpinned	test runner
fuzzywuzzy + python-levenshtein	unpinned	listed but unused (superseded by rapidfuzz)

Library	Version	Purpose
openpyxl	MISSING	needed by eval/gold/build_gold_jsonl.py; not in requirements.txt

Table 1b — Frontend (installed versions from package-lock.json; package.json declares latest):

Library	Installed version	Purpose
react / react-dom	19.2.7	UI library and DOM renderer
vite	8.0.16	dev server / build tool
@vitejs/plugin-react	6.0.2	React plugin for Vite
lucide-react	1.17.0	icon set

3. Installation and User Guide

The system runs in three modes, lightest to fullest: **CLI** (pipeline on one restaurant, no web server), **local web app** (FastAPI + React on one machine), and **hosted server** (one public URL). This section condenses `RUNNING.md`; the same commands work on Windows, macOS, and Linux.

3.1 Prerequisites

- Python 3.10+ (the codebase is verified on 3.10–3.13).
- Node.js 18+ and npm.
- An LLM provider: a Google AI Studio (Gemini) API key for the cloud profile, or Ollama with a pulled model (`ollama pull llama3.1`) for the local profile — no API key needed locally.
- The Yelp Open Dataset JSON files placed in `backend/data/raw/` (git-ignored, not shipped): `yelp_academic_dataset_business.json` and `yelp_academic_dataset_review.json`. The offline tests and the evaluation fixture run without them.

3.2 Backend Setup

```
cd backend
python -m venv venv && venv\Scripts\activate # or a conda env
pip install -r requirements.txt
copy .env.example .env # cloud profile: then set OPENAI_API_KEY=<Gemini key>
```

Configuration is entirely environment-driven:

Variable	Default	Purpose
OPENAI_BASE_URL / OPENAI_API_KEY	Gemini compat endpoint / —	provider endpoint and credential (Section 2.5)
OPENAI_MODEL / OPENAI_FALLBACK_MODEL	gemini-2.5-flash / gemini-3.5-flash	primary and fallback models
MAX_REVIEW_SAMPLE	100	review cap per restaurant, validated 1–100
RANDOM_SEED	42	reproducible review sampling
ANALYSIS_BATCH_SIZE	10	reviews per Analysis Agent LLM call

Variable	Default	Purpose
YELP_BUSINESS_PATH / YELP_REVIEW_PATH / YELP_DB_PATH	repo-relative defaults	dataset and index locations

3.3 Frontend Setup

```
cd frontend
npm install
npm run dev    # Vite dev server on http://localhost:5173
```

The frontend calls the backend at `http://localhost:8000` by default; set `VITE_API_BASE_URL` only if the backend lives elsewhere. `npm run build` produces the production bundle.

3.4 Running the System (CLI / Web / Server)

CLI (fastest check):

```
python run_pipeline.py --name "McDonald's" --pick 1 --dump-stages out/
cat out/_summary.json    # run_config confirms which provider ran
```

Flags: `--pick N` auto-selects the Nth fuzzy match, `--sample-size N` (1–100) bounds the sample, `--json` prints the raw report, `--dump-stages DIR` writes each agent's output plus `_summary.json` — the input format the evaluation harness consumes (a deliberate decoupling: any run can later be evaluated).

Local web app: terminal A runs `uvicorn app.main:app --reload --port 8000` from `backend/`; terminal B runs `npm run dev` in `frontend/`. Open `http://localhost:5173`, search a restaurant, confirm the branch, then either request a synchronous report or watch the live SSE pipeline in the Pipeline Monitor.

Hosted server (the ideal demo): build the frontend single-origin (`VITE_API_BASE_URL="" npm run build`) so static files call `/api/...` relatively, run `uvicorn --host 0.0.0.0`, and front it with a reverse proxy or a tunnel (cloudflared/ngrok) for HTTPS. Project-specific gotchas: the SSE stream stalls unless proxy response buffering is disabled (`nginx proxy_buffering off;`); a fresh host has no dataset (upload the SQLite index); a hosted box naturally demos the cloud profile — the local-LLM demo is shown on a laptop; and given the API's local-demo security posture (Section 7.3), a public URL should not be left up unattended.

3.5 Building the SQLite Index

```
cd backend
python scripts/build_db.py          # one-time, ~2-3 min; writes data/processed/yelp.db
python scripts/build_db.py --rebuild # replace an existing DB
```

Without the index every report request scans the 5.34 GB review file (one measured raw lookup took 77.5 s); with it, lookups are keyed on `business_id` and take milliseconds. After building, the raw JSON files may even be deleted — the app reads everything from the DB, falling back to a raw scan only when the DB is absent. Known limitation: the builder writes directly to the final path, so an interrupted build can leave a partial file that the loader treats as valid — rebuild with `--rebuild` if in doubt (Section 7.2).

4. Implementation Details

4.1 Member 1. Hoang Huu Hoan:

Owner: branch eval/tier2-full-73-gold; the Strategy Agent, backend/eval/, and core graph/orchestration contributions.

Served as the technical lead and project manager for the evaluation framework and overall system integration, coordinating architectural discussions, defining the evaluation methodology, and aligning contributions across team members to ensure a consistent direction.

Strategy Agent: converts the reasoning output's patterns and root causes into prioritised, traceable recommendations (each with an issue, a concrete action, a category, and an expected impact), reusing the shared BaseAgent retry loop and offering a deterministic use_llm=False mode for offline tests; priority ordering follows pattern frequency, so the recommendations the owner sees first are the ones grounded in the most reviews.

Built the four-tier evaluation methodology end to end (Section 5): Tier 1 deterministic consistency checks over stage dumps; Tier 1b reproducibility / latency / cost / failure-injection harness; Tier 2 gold-set scoring (sklearn accuracy and macro-F1) with the labelling worksheet and build_gold_jsonl tooling; Tier 3 LLM-as-judge runner with written rubrics — diagnosing the deprecated default judge (gemini-2.5-pro returning 404) and repointing it to a distinct, stronger gemini-pro-latest. Evaluation consumes the --dump-stages JSON artifacts rather than re-running the pipeline, so any past run can be evaluated after the fact. Hand-labelled all 73 LOVE Grille reviews (initial stratified 40, then extended to the full business so the sample cannot be cherry-picked) and wrote docs/EVALUATION_REPORT.md consolidating the live four-tier results. Beyond evaluation, authored the 13-finding project audit and the original hub redesign proposal (docs/ORCHESTRATOR_DESIGN.md) that framed the orchestrator discussion, contributed to the app/core/ graph and recovery logic alongside Members 1 and 3, and prepared the report skeleton plus the QA/novelty/research-grounding dossiers that ground this report's claims.

4.2 Member 2. Pham Ho Quang Dung:

Owner: dung/analysis-reasoning-agent; reasoning_agent.py, the BaseAgent self-correction loop, and the simple-hub supervision module (design and implementation, commit 953a6bd).

Reasoning Agent: pattern detection and root-cause hypotheses over the aggregated analysis rows, with contract-enforced evidence lists (evidence_review_ids min length 1) and calibrated confidence labels. BaseAgent self-correction: one shared retry loop inherited by all four agents — JSON response mode, Pydantic validation, a correction prompt carrying the exact validation error, at most two retries, then a structured AgentError that never swallows the underlying error string — plus the correction-prompt templates and the mock-data stubs (tests/mock_data.py, test_analysis_agent.py) used for offline agent testing. Contributed the initial API wiring and first dashboard version that Member 1 later consolidated into the delivery layer.

Simple-hub supervision (Option B): authored the design document (docs/ORCHESTRATOR_SIMPLE_HUB.md) weighing it against the team's full validator-node redesign — including three defects found in the original design — then implemented the full mechanism: supervision.py (measure/decide with thresholds as module constants and 23 unit tests fixtured on real run dumps), the hub wiring in graph.py with loop-safety analysis (recursion_limit), the retry-feedback mechanism, the frequency recompute-and-overwrite that makes Python authoritative over the model's claimed statistics, the new SSE event contract (verdict events, (stage, attempt) keys) handed to the frontend as a written spec, and the rewritten 8-scenario degradation harness. Verified the hub against live data (73-review LOVE Grille run, 22/22 Tier 1 checks, zero retries) and documented the verification procedure in docs/RUN_TESTS.md for the rest of the team.

Simple-hub supervision: weighing it against the team's full validator-node redesign, then implemented supervision.py (measure/decide, thresholds as module constants, 23 unit tests fixtured

on real run dumps), the hub wiring in graph.py, the retry-feedback mechanism, and the frequency recompute-and-overwrite that makes Python authoritative over the model's claimed statistics.

4.3 Member 3. Nguyen Nam Khanh:

Owner: branch feature/khanh-data; backend/app/data/ (loader, matching, preprocessor), scripts/build_db.py, and the Analysis Agent.

Analysis Agent: per-review sentiment (positive/negative/neutral/mixed) and aspect labels across seven categories. Designed the batched-classification mode (10 reviews per LLM call, with the AnalysisBatchOutput contract) that keeps the request count inside provider rate limits, with a cross-record batch validator — returned review IDs must equal the input IDs in order, or the batch re-enters the correction loop; on total batch failure it emits one AgentError per review so downstream counts stay aligned. Also added the token-usage accounting hooks that the evaluation harness uses to report LLM cost per 100 reviews.

Data layer: loader with dual paths — SQLite-first with raw-JSON line-scan fallback — where both paths share one seeded sampling helper, so the same data, seed, and cap return identical review-ID sets, verified by a parity regression test. The ~150k-record business dataset is loaded once into a module-level cache so concurrent API searches never re-read the file. MAX_REVIEW_SAMPLE is validated 1–100 (default 100, deliberately reduced from the brief's 500-most-recent to bound LLM cost), and request-level sample_size is honoured in POST, SSE, and CLI paths. Fuzzy matching replaced the brief's fuzzywuzzy (10–15 s per query) with rapidfuzz (identical Levenshtein scores, under 1 s) plus a first-character pre-filter, added review_count and address to results for ranking and branch disambiguation, and returns all score-100 branches of a chain sorted by review count so the user picks the branch. Wrote the SQLite index builder (~2–3 minute one-time build turning the measured 77.5 s raw scan into millisecond lookups), the frozen five-column preprocessing contract with empty-text and duplicate-ID removal, and the offline data-pipeline test suite (test_unit.py, test_data_pipeline.py) covering sampling, cap validation, and raw/SQLite parity.

4.4 Member 4. Tran Manh Duc:

Owner: The Report Agent, the API layer, and frontend contributions.

Report Agent: assembles the final structured report from the upstream reasoning and strategy outputs — executive summary, key findings, root causes, prioritised recommendations, and limitations — reusing the shared BaseAgent retry loop, with a deterministic use_llm=False mode so the report contract can be tested without live calls. business_name and sample_size are overwritten from validated pipeline state rather than trusted from model output.

Built the FastAPI delivery layer end to end: /health, fuzzy business search (/api/businesses/search), the synchronous report endpoint (POST /api/reports), and the SSE streaming endpoint with per-stage progress events, all sharing one _build_report_response path so both delivery routes return identical, trusted metadata. Added the CORS middleware for the Vite dev origin and the _build_analysis_summary aggregation that feeds the dashboard's sentiment and aspect charts, so the frontend renders computed statistics rather than model-claimed ones. Contributed to the React frontend (Dashboard / Pipeline Monitor navigation and report rendering) and kept the hosted-demo path single-origin (static frontend against relative /api/...) to eliminate cross-origin issues. Enforced the AGENTS.md protected-file and contract-change rules that kept shared Pydantic contracts stable for the whole project. The CLI, the API, and the live e2e test all import the same production nodes (app/core/nodes.py via app/core/pipeline.py), so the demo, the tests, and the endpoints cannot drift apart.

5. Experimentation and Evaluation

5.1 Evaluation Methodology

The harness has four tiers, each answering a question the previous tier cannot:

- **Tier 1 — deterministic consistency (no labels, no API).** Because each stage is derivable from the previous one, correctness reduces to consistency checks that need no ground truth: schema validity, evidence-ID existence, aspect grounding, frequency recomputation, recommendation traceability, report-subset checks, completion status. Offline and CI-suitable.
- **Tier 1b — behaviour.** Reproducibility across seeded runs, latency and token cost, and eight failure-injection scenarios exercising every orchestrator recovery route.
- **Tier 2 — objective accuracy.** The Analysis Agent (the only stage with objectively checkable answers) scored against a human gold set with accuracy and macro-F1 — macro-averaging chosen because it weights all classes equally under imbalance [21].
- **Tier 3 — subjective usefulness.** The stages with no formula (root causes, recommendations, report) scored 1–5 against written rubrics by an **independent, stronger judge model** — LLM-as-judge in the sense of [10][11], with a different model from the pipeline to mitigate self-preference bias [13].

Test subject: **LOVE Grille**, Philadelphia (business_id 4Env6uGYxMhXFKPfCuzUuQ, 3.0★), all **73** of its reviews (below the 100 cap, so no sampling noise). Pipeline model gemini-2.5-flash; judge gemini-pro-latest. The methodology was fixed early (decision log, 2026-06-13) and executed live end-to-end on 2026-07-15, closing the harness's own "never run live" blocker; the gold set was extended from the stratified 40 to the full 73 in the same pass. The composition of these four tiers into one harness is the team's own methodology — each tier individually follows established practice, but the bundle is not a named method from the literature (Section 7.5).

5.2 Tier 1 — Deterministic Checks

Result: 24 / 25 checks passed on the live 73-review run (committed artifact backend/out/tier1_report.json). The committed offline fixture passes 16/16, and a later live run of the hub pipeline on a different provider passed 22/22 (backend/out_hub/).

The single failure is the healthiest possible outcome: the Reasoning Agent cited evidence ID SKXs-JiPXpVnAwcXhA5wA, which does not exist — the real ID is SKXs-JiPX3pVnAwcXhA5wA; the model dropped one character while copying a 22-character Yelp ID. The evidence-existence check scored 27/28 valid IDs and failed that pattern exactly as designed. The defect is real but cosmetic (the pattern's aspect and recomputed frequency remain correct), and a clean 25/25 would arguably have been more concerning — catching one honest error with zero false positives is evidence the harness tests something. All frequency recomputations agreed with claims within ± 0.05 (e.g. claimed 0.397 vs recomputed 0.40); all 5 strategy recommendations traced to upstream causes; the report was a faithful subset of upstream outputs.

5.3 Tier 2 — Analysis Gold Set

All 73 LOVE Grille reviews were hand-labelled for sentiment and aspect mentions (first a 40-review sample stratified across star ratings, then extended to the complete business — full coverage removes any cherry-picking concern). Against these gold labels:

Metric	Score
Sentiment accuracy	0.753 (55/73)
Aspect macro-F1	0.852

Per-aspect F1 (binary "is this aspect mentioned"), with gold support:

Aspect	F1	Gold support (mentions)
cleanliness	1.000	4 (low support — indicative only)
food_quality	0.975	59
staff_attitude	0.966	43
pricing	0.958	34
wait_time	0.905	22
other	0.734	35
ambiance	0.429	3 (low support — indicative only)

Aspect detection is strong on every well-supported category (≥ 0.905 wherever support ≥ 22). The two extremes — cleanliness at a perfect 1.000 on 4 mentions and ambiance at 0.429 on 3 — are both low-support artifacts where a single disagreement swings F1 by 0.3+, and are reported with that footnote rather than as headline results. Sentiment accuracy at $n=73$ (0.753) was essentially unchanged from $n=40$ (0.750), retroactively confirming the stratified 40 was representative. The gold set has a single annotator; inter-annotator agreement (Cohen's κ) is the one remaining gap for a fully defensible Tier 2 number (Section 8.3).

5.4 Tier 3 — LLM-as-Judge (Rubrics)

An independent, stronger judge (gemini-pro-latest — deliberately different from the pipeline's gemini-2.5-flash to avoid self-judging [13]) scored each subjective output 1–5 against fixed written rubrics (committed artifact backend/out/tier3_scores.json):

Dimension	Mean (1–5)	n	Notes
Root-cause plausibility	4.8	5	four 5s; the lone 4 was an inherently generic "unspecified negative experiences" cause
Recommendation actionability	4.2	5	four 4s; the single 5 (QR-code feedback system) was praised as implementable "within a week"
Report usefulness	5.0	1	"findings/root causes/recommendations logically linked... limitations clearly and honestly stated"

The judge reproduced an earlier scoring run exactly (4.8 / 4.2 / 5.0), a positive reliability signal. These scores correspond to the pre-hub Gemini run; re-judging a current hub-pipeline dump would keep Tier 3 aligned with the architecture described in Section 2 (advisable, not blocking).

5.5 Latency and Cost

Stage	Time (73 reviews)
analysis	192.06 s (~74% of wall time)
reasoning	31.12 s
strategy	16.82 s

Stage	Time (73 reviews)
report	20.34 s
total	260.33 s → 356.62 s / 100 reviews
tokens	108,270 → 148,315 / 100 reviews

Analysis dominates because it is per-review LLM classification in batches of 10; it is the only stage worth optimising (larger batches or concurrency) if demo latency matters. Reproducibility across two seed-42 runs: PASS. Failure-injection: **8/8** scenarios pass in the current harness (healthy proceed, low-confidence flagging, insufficient-data halt, critical-majority-failure retry, critical-exhausted halt, fabricated-evidence retry, non-critical give-up on record, report give-up halt). Earlier documentation cites 6/6 — that count predates the hub's two additional scenarios.

6. Demonstration Scenarios: End-to-End Report for a Restaurant

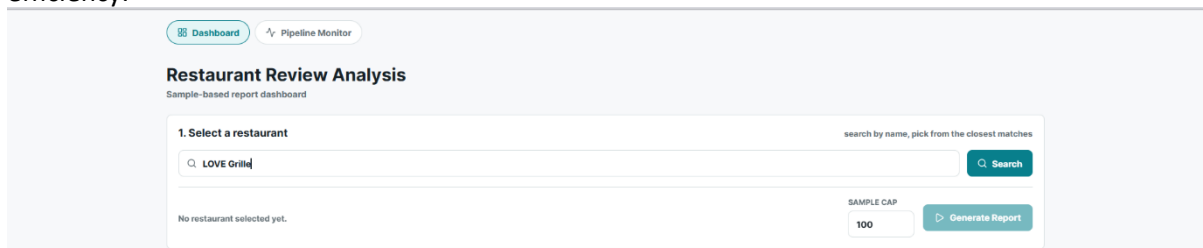
The canonical demo: search "LOVE Grille", confirm the Philadelphia branch from the fuzzy-match candidates, and stream the report. The committed artifacts of exactly this run exist in backend/out/ — 73/73 successful analyses, 5 patterns with evidence IDs, 5 root causes, 5 prioritised recommendations, and a final report whose top recommendation (a QR-code table-feedback system) was the single item the Tier 3 judge scored 5/5 for actionability. Pipeline status complete: 0 agents skipped, 0 failures, 0 retries.

6.1 User flow on how to use the web

Step 1 Search the restaurant for analyze

The first step of the system requires the user to input the name of the restaurant they wish to analyze. As illustrated in the figure, the interface provides a search bar where users can type the restaurant name — in this case, "LOVE Grille" — and click the **Search** button to look up matching businesses from the Yelp Academic Dataset.

The system performs a name-based lookup across the dataset's business records, returning the closest matches for the user to select from. Once a restaurant is selected, the user can configure the Sample Cap parameter, which controls the maximum number of reviews to be retrieved and analyzed. By default, this value is set to 100 reviews, balancing analytical depth with processing efficiency.



The screenshot shows a web interface titled "Restaurant Review Analysis" with a subtitle "Sample-based report dashboard". At the top, there are two tabs: "Dashboard" (active) and "Pipeline Monitor". The main section is titled "1. Select a restaurant". It features a search bar with the text "LOVE Grille" entered. To the right of the search bar is a "Search" button. Below the search bar, it says "No restaurant selected yet." To the right of the search bar, there is a label "search by name, pick from the closest matches". Below the search bar, there is a "SAMPLE CAP" section with a value of "100" and a "Generate Report" button.

Step 2 Choose the restaurant

After clicking the Search button, the system queries the Yelp Academic Dataset and returns the closest matching businesses. As shown in the figure, searching for "LOVE Grille" returns 1 closest match, displaying key information to help the user make an informed selection:

The user then clicks on the desired result to confirm their selection before proceeding to generate the analysis report. This step ensures that the system analyzes reviews from the correct and specific business location, avoiding ambiguity that may arise when multiple branches share the same restaurant name.

The screenshot shows the 'Restaurant Review Analysis' dashboard. At the top, there are two tabs: 'Dashboard' (active) and 'Pipeline Monitor'. Below the tabs, the title 'Restaurant Review Analysis' is followed by the subtitle 'Sample-based report dashboard'. The main section is titled '1. Select a restaurant'. On the right, it says 'search by name, pick from the closest matches'. There is a search input field containing 'LOVE Grille' and a 'Search' button. Below the input, it says '1 closest matches — pick one to continue.' A list of results is shown, with 'LOVE Grille' at '8000 Essington Ave, Philadelphia, PA' highlighted with a teal border. To the right of the result, it shows '72' reviews and '100%' completion.

Step 3 Click the generate report

Once the restaurant has been selected, the interface updates to confirm the chosen business and display its details. As shown in the figure, **LOVE Grille** at 8000 Essington Ave, Philadelphia, PA is highlighted with a teal border indicating it has been selected, along with a confirmation showing **72 reviews** available for analysis.

The user then clicks the **Generate Report** button to initiate the full multi-agent pipeline. At this point, the system sequentially executes the following agents

This screenshot shows the same dashboard as the previous one, but with the 'LOVE Grille' result highlighted with a teal border. Below the search results, there is a section for 'LOVE Grille' showing '72 reviews' and a 'Generate Report' button. The button is teal with a white play icon and the text 'Generate Report'. To the left of the button, there is a 'SAMPLE CAP' input field set to '100'.

Step 4 Output

Once the pipeline completes, the system renders a structured analytical report composed of six sections. Each section corresponds to the output of one or more agents in the multi-agent pipeline.

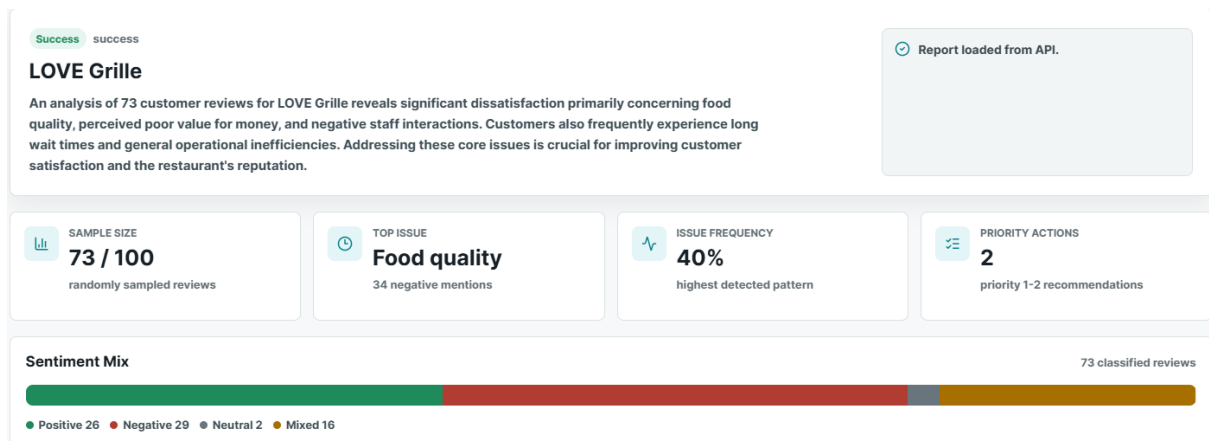
6.2 Report generated from the web

1 Executive Summary and Key Metrics

The report opens with a success indicator and an executive summary generated by the Report Agent. For LOVE Grille, the analysis of 73 customer reviews identified significant dissatisfaction centred on food quality, perceived poor value for money, and negative staff interactions, with long wait times and operational inefficiencies as secondary concerns.

Four metric cards summarise the analysis at a glance: Sample size, top issue, issue frequency, priority actions.

Below these, the Sentiment Mix bar visualises the distribution across 73 classified reviews: 26 positive, 29 negative, 2 neutral, and 16 mixed. The inclusion of a *mixed* category allows the system to capture reviews expressing both praise and criticism, which a strict three-class scheme would otherwise misclassify.



2 Key Findings and Aspect Breakdown

The Key Findings panel presents five natural-language observations extracted from the review corpus, covering food inconsistency, price–value misalignment, staff attentiveness, general dissatisfaction, and excessive waiting periods.

The Aspect Breakdown table decomposes sentiment across seven aspect categories, reporting positive, negative, and neutral counts alongside a computed main signal.

Pricing emerges as the most polarised aspect, with negative mentions outnumbering positive ones by roughly nine to one. Food quality, despite being flagged as the top issue by absolute negative volume, retains a marginally positive signal — illustrating why aspect-level counts must be read alongside the pattern analysis rather than in isolation.

Key Findings				5 findings
- Customers frequently report dissatisfaction with the quality of the food, indicating inconsistencies or general poor preparation. - A significant number of customers feel that the prices are too high for the quality or quantity of food/service received, leading to a perception of poor value. - Customers frequently report negative experiences with staff, including rudeness, unhelpfulness, or general lack of attentiveness. - A notable portion of reviews indicate a general dissatisfaction with the overall experience, often without specifying a single aspect, or mentioning multiple minor issues. - Customers frequently complain about excessive waiting periods for seating, ordering, or receiving their food.				
Aspect Breakdown				
ASPECT	POSITIVE	NEGATIVE	NEUTRAL	positive, negative, neutral labels MAIN SIGNAL
Other	17	21	6	Negative
Food quality	39	34	5	Positive
Wait time	14	9	1	Positive
Staff attitude	28	18	1	Positive
Cleanliness	2	2	0	Positive
Pricing	3	28	5	Negative
Ambience	8	1	0	Positive

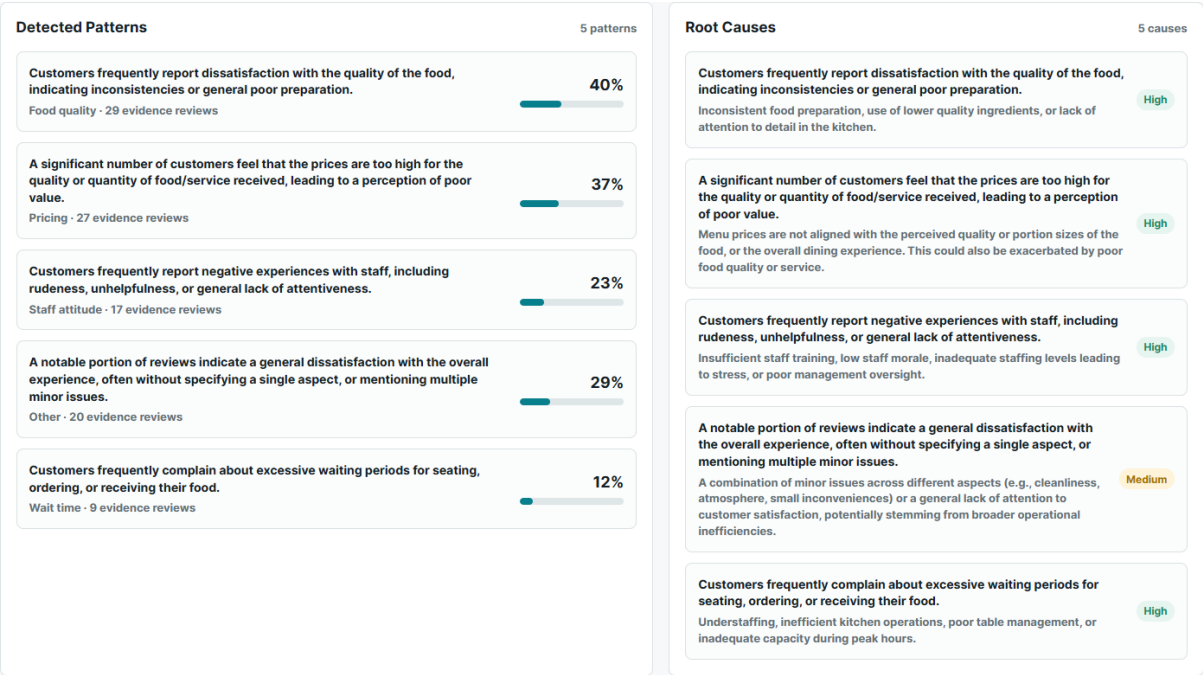
3 Detected Patterns and Root Causes

The Detected Patterns panel quantifies how frequently each issue appears across the corpus, with each pattern supported by a count of evidence reviews:

- Food quality dissatisfaction — 40% (29 evidence reviews)
- Price–value misalignment — 37% (27 evidence reviews)
- General dissatisfaction — 29% (20 evidence reviews)
- Negative staff experiences — 23% (17 evidence reviews)
- Excessive wait times — 12% (9 evidence reviews)

Adjacent to each pattern, the Root Causes panel provides an inferred explanation together with a confidence label (High or Medium). For example, food quality issues are attributed to inconsistent

preparation, lower-quality ingredients, or insufficient attention to detail in the kitchen, and are assigned High confidence. The general dissatisfaction pattern receives a Medium label, reflecting the inherent ambiguity of reviews that do not identify a specific aspect.



from free text is inherently subjective; and a sample of 73 reviews, while indicative, may not capture every nuance of customer experience. Surfacing these constraints in the output itself encourages appropriate interpretation of the findings rather than treating them as definitive.

Limitations

sampling constraints

- Analysis is based solely on publicly available customer reviews, which may not represent the entire customer base.
- Sentiment and specific issues are inferred from text, which can sometimes be subjective.
- The sample size of 73 reviews, while indicative, may not capture all nuances of customer experience.

7. Critical Analysis of the Implementation

7.1 Strengths

- **Provider-agnostic by construction.** One OpenAI-compatible abstraction, environment-selected; the same code verifiably ran on Gemini and Groq, with a wired local Ollama profile — plus per-run provenance recording.
- **Structured contracts everywhere.** Every inter-agent payload is Pydantic-validated at generation time and re-measured by the hub; evidence lists cannot be empty; limitations are schema-mandatory.
- **Bounded, feedback-carrying recovery.** Two-retry caps at both the agent and graph level, retries that carry the measured failure into the prompt, and clean halts for critical stages instead of hollow reports.
- **A four-tier evaluation harness with teeth.** Its single live failure was a genuine caught defect — the harness demonstrably detects real errors without false positives.
- **Test discipline:** 119 tests collected, 113 passing offline without any API key (6 live integration tests deselected by marker), including 23 supervision unit tests fixtured on real run dumps.

7.2 Limitations and Known Issues

- **Business identity at the API boundary:** `business_id` and `restaurant_name` are accepted independently, fuzzy matching has no minimum acceptance score, and the report endpoint auto-selects the first match when no ID is supplied — a direct API client can mislabel one business's reviews with another's name.
- **Reasoning sees statistics, not text.** The Reasoning Agent receives only review IDs, sentiments, and aspect labels — no review text, dates, or stars — so root causes are inferred from aspect statistics rather than what reviewers actually wrote, and no temporal reasoning is possible. Forwarding text and dates is the single highest-value improvement identified and deferred (Section 9.2).
- **Batch failure granularity:** one failed batch of 10 costs all 10 reviews (each becomes an `AgentError`), and analysis proceeds as long as the failure ratio is not above 50%.
- **Non-atomic index build:** an interrupted `build_db.py` leaves a partial database that the loader's existence-check accepts as valid.
- **Frontend gaps:** Dashboard and Pipeline Monitor own separate state (stale-report and duplicate-run risks); the Monitor does not yet consume the hub's `verdict/attempt` SSE events; accessibility needs hardening (focus outlines, live regions, colour-only state, some sub-AA contrast).

- **Dependency hygiene:** backend requirements unpinned and carrying two unused fuzzy-matching packages; openpyxl missing; frontend package.json declares latest (the lockfile pins actuals). No CI workflow runs any of the checks automatically.
- **Evidence gaps for this report itself:** Tier 2 and latency artifacts are prose-backed only, and the committed Gemini dump predates run_config recording (flagged inline in Sections 2.5, 5.3, 5.5).

7.3 Security and Deployment Considerations

The API is deliberately a trusted-local-network demo: wildcard CORS, no authentication, no rate limiting, no LLM cost controls, and SSE error events that relay raw exception details. None of this is hidden — it is documented posture, acceptable for a course demonstration and explicitly not a public deployment. Hardening (auth, origin restrictions, error sanitisation, quota guards) is future work rather than a claimed feature.

7.4 Design Trade-offs

- **Sampling vs completeness:** a seeded ≤ 100 -review sample keeps runs affordable and reproducible at the cost of coverage; the system compensates by stating the bound everywhere (schema-mandatory limitations, sample_size in the report) rather than pretending completeness. Note the median Yelp business has ~ 15 reviews, so for most businesses the cap is not the binding constraint — small-n is, which the hub flags (low_confidence:n<30).
- **Prompted LLM vs fine-tuned ABSA:** published evaluations find prompting generally trails fine-tuned task-specific models on ABSA [15][16]. The team chose prompting for zero training cost, provider portability, and instant taxonomy changes — a flexibility/cost decision, not an accuracy-optimal one, and the report claims it as such.
- **Deterministic vs LLM orchestration:** an LLM supervisor could route on nuance; a deterministic one is auditable, free, reproducible, and unit-testable. Given that every quantity worth routing on is already measurable in Python, the team kept the LLM out of the routine path (a logged LLM-escalation branch remains a ~ 15 -line future option).
- **Single vs multi-model:** one flash-class model for all four agents is cheap and simple; per-stage model selection (e.g. a stronger model only for reasoning) was left unexplored.
- **Dropping skip:** the redesign traded a five-verdict vocabulary for four, converting silent stage-skipping into on-record proceed_with_warning + flags — less magic, more traceability.

8. Discussion of Experimental Results

8.1 Interpretation of Results

Read together, the four tiers support four distinct claims. **Honesty** (Tier 1): the pipeline's claims are internally consistent and grounded — every cited evidence review genuinely carries the claimed aspect, and claimed frequencies match recomputation; the one failure was a caught single-character transcription slip, which is the harness working, not failing. **Robustness** (Tier 1b): identical outputs across seeded runs and correct behaviour on all eight injected failure routes. **Accuracy** (Tier 2): 0.852 aspect macro-F1 on the full 73-review business, with every well-supported aspect ≥ 0.905 .

Usefulness (Tier 3): 4.8 / 4.2 / 5.0 from an independent stronger judge.

The gap between aspect macro-F1 (0.852) and sentiment accuracy (0.753) is informative rather than alarming: aspect mention is close to an objective property of the text, while overall sentiment of mixed and neutral reviews is genuinely subjective — the ~ 18 disagreements cluster exactly there.

This is why sentiment is the only field treated as objectively scorable and the later stages are evaluated by rubric instead. The stability of sentiment accuracy from $n=40$ (0.750) to $n=73$ (0.753) suggests the metric is not sample-artefact noise.

The judge's per-item comments also localise the system's quality ceiling: the weakest root cause was the inherently generic "unspecified negative experiences" (scored 4), and recommendations lost points mainly for leaving ownership details to the operator — both consistent with the Reasoning Agent's input being aspect statistics rather than review text (Section 7.2), which bounds how specific its explanations can be.

8.2 Threats to Validity

- **Single restaurant, single domain:** all live numbers come from one 73-review Philadelphia restaurant; they are indicative, not population estimates.
- **Single annotator:** the Tier 2 gold set has no second annotator, so no inter-annotator agreement (Cohen's κ) can be reported; label subjectivity directly bounds the sentiment-accuracy ceiling.
- **LLM-judge biases:** LLM judges exhibit position bias [12] and self-preference [13]; the design mitigates self-preference with an independent judge family but rubric scoring of single outputs (no pairwise comparison) leaves verbosity and leniency biases unquantified; the report-usefulness score rests on $n=1$.
- **Provenance gaps:** Tier 2 scores and latency/cost figures are documented in prose but their backing artifacts are not committed; the Gemini dump lacks a recorded `run_config` (flagged in Sections 2.5/5.3/5.5 with the exact regeneration steps).
- **Sampling noise:** not a factor for the 73-review test case (below the cap), but for large businesses a different seed samples different reviews; reproducibility is guaranteed per seed, not across seeds.
- **Evaluation-supervision coupling:** the hub enforces the same checks Tier 1 measures, so post-hub Tier 1 pass rates partly measure the supervisor's own filtering — an inherent property of promoting evaluation into orchestration, worth stating when comparing pre- and post-hub numbers.

9. Beyond The Class Project Scope

The project was extended beyond the original coursework by incorporating additional model benchmarking and system architecture evaluations, significantly improving its technical depth and research contribution. These extensions transformed the initial class project into research-oriented work. The first draft of the research paper, including detailed methodology, experiments, and results are presented in canvas submission file

"Evaluation_as_Supervision__Deterministic_Runtime_Verification_of_Claims_in_a_Multi-Agent_LLM_Pipeline.pdf".

The paper will be revise to submit to **RIVF 2026**.

10. Summary and Conclusion

10.1 Summary of Achievements

Against G1, the project delivered autonomous LLM agents with schema-validated output and bounded self-correction. Against G2, it delivered a four-agent collaborating system under a

deterministic supervising hub that measures every stage, corrects fabricated statistics, retries with feedback, and halts honestly — implemented in LangGraph, delivered through FastAPI and a React dashboard with live progress streaming, and running unchanged on cloud and local LLM providers. The system was evaluated live on real Yelp data across four tiers: 24/25 deterministic checks (the single miss being a genuinely caught LLM defect), reproducible behaviour with 8/8 failure-injection scenarios, 0.753 sentiment accuracy and 0.852 aspect macro-F1 against a fully hand-labelled 73-review gold set, and 4.8 / 4.2 / 5.0 rubric scores from an independent judge. 113 offline tests pass without any API key.

10.2 Future Work

Forward review text and dates to the Reasoning Agent — the single highest-value quality fix: root causes grounded in what reviewers wrote, and temporal patterns (e.g. decline after a date) become possible.

Business-identity hardening: a minimum fuzzy-match acceptance score and ID/name consistency validation at the API boundary.

Index-build atomicity (write to temp path, validate, atomic rename) and a schema/completeness check in the loader.

CI: run the offline suite and Tier 1 fixture on every push; pin dependencies.

Second annotator + Cohen's κ for the gold set; extend the gold set to a second restaurant to test generalisation.

Frontend: consume verdict/attempt SSE events, share report state across views, accessibility hardening.

Richer evaluation: confidence intervals on frequencies, relevance filtering (n_{relevant}), Tier 3 on the hub pipeline, and pairwise judge comparisons to control position bias [12].